BRICK-OF-TICKS STATISTICAL EDGE SYSTEM

Production-Ready Implementation Plan v6

OTC XAUUSD (Gold) | Level 1 Quote Data | Renko Brick Framework

Phase 1: The Atomic Physics Engine

Objective: Transform raw dealer quotes into a stationary, 9-dimensional pressure vector that captures hidden liquidity dynamics without mathematical singularities or look-ahead bias.

1.1 Input Stream

You receive a continuous stream of Level 1 dealer quote updates from your broker. Each tick is a tuple:

```
T_k = { t_k, P^B_k, P^A_k, q^B_k, q^A_k }
```

where t is timestamp (milliseconds), P^B is bid price, P^A is ask price, q^B is bid volume, q^A is ask volume.

Critical Note: For OTC XAUUSD, there is NO separate trade price. You receive quote updates only. Lee-Ready tick classification, which requires a distinct trade price, is therefore invalid for this data. The OFI formula below is derived entirely from quote changes and requires no trade price.

1.2 The 9-Dimensional Micro-Vector (v_k)

Calculated on every single incoming tick. Do not batch. Do not wait for a brick to close. All features are computed sequentially from T_k and its predecessor T_{k-1}.

#	Feature	Corrected Formula / Logic	Source / Purpose

1	z_OFI	`e_k = I(dBid>=0)*q^B_k - I(dBid<=0)*q^B_{k-1}`
		`- I(dAsk<=0)*q^A_k + I(dAsk>=0)*q^A_{k-1}`
		`z_OFI = (e_k - mu_1000) / sigma_1000`
			Cont et al. Core pressure signal. WEAK inequalities (>= and <=) capture limit order refreshes when price is static (dP=0). Strict inequalities would discard this signal. No trade price needed.

#	Feature	Corrected Formula / Logic	Source / Purpose

2	z_Depth	`D_k = q^B_k + q^A_k`
		
		`z_Depth = (D_k - mu_1000) / sigma_1000`
			Cont et al. Total liquidity at the touch. Low depth = thin market = price moves easily. High depth = absorption wall = momentum likely to stall.

3	z_Susc	`S_raw = e_k / (D_k + 1e-8)  [divide RAW values first]`
		
		`z_Susc = (S_raw - mu_1000) / sigma_1000`
			Cont et al. Susceptibility: pressure per unit of resistance. CRITICAL: divide raw values then z-score. Never divide two z-scored values (causes near-zero denominator singularities).

4	z_Vel	`V_k = 1 / (t_k - t_{k-1} + 1e-3)  [t in milliseconds]`
		
		`z_Vel = (V_k - mu_1000) / sigma_1000`
			Ait-Sahalia. Tick arrival velocity. High velocity = high volatility. Inverse of duration avoids extreme right skew of raw duration values.

5	z_Spread	`S_k = P^A_k - P^B_k`
		
		`z_Spread = (S_k - mu_1000) / sigma_1000`
			Rayment. Dealer bid-ask spread. Sudden spread widening signals dealer risk aversion or uncertainty (e.g., news events). A reversal risk indicator.

6	Progress	`P_mid_k = (P^A_k + P^B_k) / 2`
		
		`Progress = (P_mid_k - P_brick_open) / brick_size`
			Rayment. Price location within current brick. Produces a sawtooth wave across the rolling buffer: each brick reset creates a discontinuity the CNN can detect. Also encodes brick velocity implicitly (fast bricks = rapid resets).

7	Flag_Curr	`1 if tick belongs to the currently-closing brick`
		
		`0 if tick is spillover from a prior brick`
			Leakage Guard (structural). Allows the CNN to learn to weight current-brick ticks differently from spillover ticks. Prevents the model from treating all 100 ticks as equally reliable.

8	Flag_Zone	`1 if |P_mid_k - P_open_of_prev_brick| >= brick_size`
		
		`0 otherwise`
			Leakage Guard (surgical). Flags ticks that occurred AFTER the previous brick's TP or SL was theoretically hit. These ticks carry price-reaction echo, not precursor physics. Model learns to discount them.

#	Feature	Corrected Formula / Logic	Source / Purpose
9	Decay	`Decay = (CurrentBrickID - TickBrickID) / MaxBufferDepth`	Attention gradient. Assigns higher values to older (less trusted) spillover ticks. Range: 0.0 (current brick) to 1.0 (oldest tick in buffer). Continuous version of Flag_Curr; provides fine-grained recency weighting.

1.3 The 3-Dimensional Macro-Vector (M_t)

Calculated only at brick close. One macro-vector is stored per brick in the Macro-History buffer (Phase 2.2).

#	Feature	Formula	Rationale
1	Log_Dur	log(brick_duration_seconds + 1)	Strongest single predictor in the dataset. Bricks <30s: 72.5% WR. Bricks >1hr: 17% WR. Log transform handles extreme right skew (max duration = 7,314 minutes).
2	Direction	+1 if green brick, -1 if red brick	Encodes the outcome of this brick as a signed directional signal for the LSTM sequence.
3	z_Size	(brick_size_t - mu_50bricks) / sigma_50bricks	Corrected. brick_size/ATR is near-constant in Renko (8% CoV) and carries no information. Deviation from the 50-brick rolling mean captures whether the current brick is anomalously large or small relative to recent norms.

Why No Trade Imbalance (TI) Feature?: TI was removed entirely. Lee-Ready, which is required to compute TI, is invalid for OTC quote-only data. The information TI was intended to carry is fully captured by z_OFI, which measures directional pressure from quote changes directly with greater precision.

Phase 2: State Persistence and Buffer Management

Objective: Maintain the rolling feature buffers and z-score normalization state across ticks and across session restarts, preventing cold-start errors and training/inference mismatches.

2.1 Z-Score Normalization State Persistence

The plan specifies rolling z-scores over the last 1,000 ticks. This is a ROLLING window, not a cumulative one. Welford's algorithm computes cumulative statistics over all ticks ever seen and must NOT be used here.

Correct Persistence Method: Use a collections.deque(maxlen=1000) per feature to store raw values. At session shutdown, serialize all five deques to disk (state.json). At session startup, load the deques and recompute mean/std from the loaded values. This guarantees the z-score basis is always the correct rolling window.

Startup Procedure

- Load state.json if it exists. Reconstruct deque(1000) for each of the 5 raw features: [e_k, D_k, S_raw, V_k, S_k].
- If state.json does not exist (first run): start with empty deques. No z-scores are computed until each deque reaches N >= 30. Features are set to 0 until then.
- After loading: recompute mu = mean(deque), sigma = std(deque) for each feature. These are the initial normalization parameters.

Per-Tick Update (Welford-Style Online Update for Efficiency)

Rather than recomputing mean/std from scratch on every tick, use an incremental update. When the deque is full (N=1000), the outgoing value (x_old) and incoming value (x_new) allow an O(1) update:

```
mu_new = mu_old + (x_new - x_old) / N
M2_new = M2_old + (x_new - x_old) * ((x_new - mu_new) + (x_old - mu_old))
sigma = sqrt(M2_new / (N - 1))
```

This is the correct hybrid: a sliding-window online update. Maintain {mu, M2} alongside the deque. Reset M2 on startup by computing it from the loaded deque contents.

2.2 The Micro-Buffer (Tick-Level Rolling Window)

- Structure: collections.deque(maxlen=100). Stores 9-dimensional vectors v_k.
- Update: Append v_k on every tick. Never cleared. Never reset at brick boundaries.
- Purpose: Captures the last 100 ticks of microstructure physics regardless of which brick(s) they belong to. A fast brick (10 ticks) will have 10 current ticks and 90 spillover ticks. The CNN flags (Flag_Curr, Flag_Zone, Decay) allow the model to distinguish them.

2.3 The Macro-History Buffer (Brick-Level Rolling Window)

- Structure: A list of the last 10 closed bricks. FIFO: oldest brick drops off when an 11th is added.
- Storage per brick: Store the RAW (100, 9) numpy array (snapshot of the micro-buffer at brick close) AND the 3-dimensional macro-vector M_t. Do NOT store the CNN embedding.

Why Raw Snapshots, Not Embeddings?: The CNN embedding is computed by the model with batch normalization statistics. If you pre-compute and store embeddings brick-by-brick (single-sample inference), the normalization statistics differ from those used during training (full-batch). Storing raw snapshots and reconstructing the (10, 100, 9) tensor at prediction time ensures the entire forward pass runs identically in training and live trading.

- Memory cost: 10 bricks x 100 ticks x 9 features x 4 bytes = 36 KB per prediction. Negligible.
- Shutdown: Serialize the Macro-History list to disk alongside the z-score deques.

2.4 Brick Identification and Metadata

Maintain a global integer BrickID that increments by 1 each time a brick closes. Each tick pushed into the micro-buffer receives the CurrentBrickID at the time of insertion. This ID is used to compute Flag_Curr (tick.BrickID == CurrentBrickID) and Decay ((CurrentBrickID - tick.BrickID) / 10).

Phase 3: The Hierarchical Model Architecture

Objective: Build a CNN+LSTM hybrid that zooms in on tick-level physics (CNN) and zooms out on the 10-brick evolution of pressure and exhaustion (LSTM), with corrected pooling to preserve temporal order.

3.1 Model Inputs

Input Name	Shape	Content
Micro Tensor	(Batch, 10, 100, 9)	For each of the last 10 bricks: the 100-tick micro-buffer snapshot as a (100, 9) array of z-scored feature vectors.
Macro Tensor	(Batch, 10, 3)	For each of the last 10 bricks: the [Log_Dur, Direction, z_Size] macro-vector computed at that brick's close.

3.2 Layer 1: Time-Distributed CNN (The Microscope)

The same CNN is applied independently to each of the 10 brick snapshots using Keras TimeDistributed wrapper. The CNN extracts a 32-dimensional embedding from each (100, 9) slice.

CNN Block (applied to each brick independently via TimeDistributed)

```
Input:  (100, 9)  [100 ticks, 9 features]

  → Branch A: Conv1D(filters=16, kernel=1, padding='causal') + LeakyReLU(0.1)
[instant imbalance]

  → Branch B: Conv1D(filters=16, kernel=3, padding='causal') + LeakyReLU(0.1)
[micro-momentum]

  → Branch C: Conv1D(filters=16, kernel=5, padding='causal') + LeakyReLU(0.1)  [local
trend]

  → Concatenate branches along feature axis  ->  (100, 48)

  → MaxPool1D(pool_size=4)  ->  (25, 48)  [preserves temporal position, keeps
strongest signal]

  → Flatten()  ->  (1200,)  [retains positional information for Dense]

  → Dense(32, activation='relu')  ->  (32,)  [compresses to brick embedding]

  → Dropout(0.3)

Output: (32,)  per brick  ->  TimeDistributed output: (10, 32)
```

Why MaxPool1D not GlobalAvgPool?: GlobalAvgPool averages all 100 timesteps equally, destroying temporal order. Tick 1 and Tick 100 contribute identically. MaxPool1D(4) reduces 100 positions to 25 by taking the maximum value in each pool, preserving the strongest activation at each temporal position. The model can then distinguish 'OFI spiked in the last 10 ticks' from 'OFI spiked in the first 10 ticks'.

Why Causal Padding?: Causal padding ensures the convolution at position t only uses information from positions t and earlier, not future ticks. This is correct for financial time series and eliminates look-ahead within the buffer.

3.3 Layer 2: Fusion (The Context Merger)

For each of the 10 bricks, concatenate the CNN embedding (32-dim) with the brick's macro-vector (3-dim) along the feature axis.

```
Fused_t = Concatenate([CNN_embedding_t, Macro_vector_t])  ->  (35,) per brick

Full sequence: (10, 35)
```

3.4 Layer 3: LSTM Core (The Telescope)

The LSTM receives the sequence of 10 fused vectors and learns the temporal evolution of pressure across bricks: e.g., high OFI 3 bricks ago + decreasing velocity + increasing duration = exhaustion pattern.

LSTM Core

```
Input:  (10, 35)  [sequence of 10 fused brick representations]
  → LSTM(units=32, return_sequences=False)
  → Dropout(0.3)
Output: (32,)  single vector representing state after processing all 10 bricks
```

Why LSTM(32) not LSTM(64)?: The training set has approximately 24,578 samples. LSTM(64) with the fusion layer produces ~33,000 parameters in this layer alone. At a 0.7x samples-to-params ratio the model will overfit. LSTM(32) keeps total model parameters around 18,000-22,000, giving a workable ratio with Dropout regularization.

3.5 Layer 4: Dual Prediction Heads

Head A: Existence Classifier

```
Input:  (32,)  from LSTM
  → Dense(1, activation='sigmoid')
Output: P(Win): Probability that price hits TP before SL. Range [0.0, 1.0]
```

Head B: Magnitude Regressor

```
Input:  (32,)  from LSTM
  → Dense(1, activation='relu')
Output: Pred_OS: Predicted overshoot ratio = (price_peak - brick_close) / brick_size
```

3.6 Parameter Budget

Layer	Parameters (approx)	Notes
3x Conv1D branches (kernels 1,3,5)	~870	3 x (9x16 + 16)

Layer	Parameters (approx)	Notes
Dense(32) after MaxPool+Flatten	~38,400	1200 x 32 + 32
LSTM(32)	~8,700	4 x (35+32) x 32
Dual heads	~66	32+1 each
TOTAL	~48,000	Training samples: 24,578. Ratio: ~0.5x. Requires Dropout(0.3) + L2(1e-4).

Regularization Required: With a sample/parameter ratio below 1.0, regularization is essential. Apply Dropout(0.3) after the CNN Dense layer and after the LSTM. Apply L2 weight decay (lambda=1e-4) to all Dense and LSTM kernels. Use early stopping with patience=15 on validation loss.

Phase 4: Training Strategy

Objective: Train the model on historically accurate data with a walk-forward split, corrected loss masking, and appropriate exclusions to prevent the model from learning leakage artifacts.

4.1 Walk-Forward Data Split

Critical Rule: Never use a random train/test split for financial time series. A random split allows future data to contaminate the training set (look-ahead bias). Always split strictly by time.

Split	Date Range	Purpose	Size (approx)
Train	Jan 2020 - Dec 2022	Model weight learning	~24,578 bricks after exclusions
Validation	Jan 2023 - Mar 2023	Threshold calibration (Prob_Win cutoff, Mag cutoff). Tune here.	~1,200 bricks
Test	Apr 2023 - Dec 2023	Final evaluation only. Never open until all hyperparameters locked.	~3,600 bricks

Win rate drifts year over year in the dataset (2020: 48.0%, 2021: 49.4%, 2022: 50.9%, 2023: 51.8%). A random split would mix 2023 regime data into training, inflating apparent performance.

4.2 Training Exclusions

- Exclude all bricks with Duration < 2 seconds from the training set. These bricks are handled by the hard rule (Phase 5, Step 2) and their micro-buffers are approximately 95% spillover. Training on them teaches the model leakage patterns rather than genuine physics.
- Do NOT exclude these bricks from the validation and test sets. Their outcomes are needed for overall performance evaluation.
- Bricks in consecutive fast-brick chains of depth > 5: Apply sample_weight = 0.5 in the loss function. These bricks have deeply tainted spillover buffers (spanning multiple completed trades). Do not exclude them entirely, but reduce their gradient influence.

4.3 Label Construction for Head B

For each brick in the training set, construct the dual target:

Outcome	y_class (Head A target)	y_mag (Head B target)
WIN (price hit TP)	1	actual_overshoot_ratio = (price_peak - brick_close) / brick_size
LOSS (price hit SL)	0	0.0 (the correct prediction for a losing trade is zero magnitude)

Why y_mag = 0.0 for LOSSes (not masking)?: The previous plan masked Head B on LOSS samples. This is wrong: Head B never sees patterns that look bullish but ultimately fail. It becomes systematically overconfident. Training it to output 0.0 for LOSSes teaches the regressor that high-OFI patterns which still lose should produce zero predicted magnitude. This is the correct expected-value framing.

4.4 The Hybrid Loss Function

```
L_total = alpha * BCE(p_hat, y_class) + beta * Huber(m_hat, y_mag, delta=1.0)

alpha = 1.0   [classification loss weight]

beta  = 0.3   [regression loss weight - lower because regression is noisier]

delta = 1.0   [Huber threshold - robust to outlier overshoot values]
```

Huber loss is chosen over MSE for Head B because overshoot values have a heavy right tail (some bricks move 3-5x brick_size). MSE would over-penalize these rare large moves, making the regressor systematically conservative.

The alpha:beta ratio determines training priority. Starting values: alpha=1.0, beta=0.3. Tune on validation set if one head consistently underperforms.

4.5 Training Configuration

Parameter	Value	Rationale
Optimizer	Adam(lr=1e-3)	Standard. Reduce to 1e-4 after 20 epochs if loss is noisy.
Batch size	64	Balances gradient stability with training speed for ~24k samples.
Max epochs	200	With early stopping, actual training typically 40-80 epochs.
Early stopping	patience=15 on val_loss	Prevents overfitting. Restore best weights.
LR schedule	ReduceLROnPlateau(factor=0.5, patience=8)	Halves learning rate if val_loss stagnates.
Dropout	0.3 after CNN Dense and LSTM	Primary regularization given tight sample/param ratio.
L2 weight decay	1e-4 on Dense and LSTM kernels	Secondary regularization.
Chain weight	0.5 for chain_depth > 5	Reduces gradient influence of deeply tainted samples.

4.6 Threshold Calibration

The execution filters (Prob_Win > 0.60 and Pred_OS > 1.1) must be calibrated on the VALIDATION SET, not chosen a priori. After training:

- Plot precision-recall curve on validation set for Head A. Choose the Prob_Win threshold that gives the desired precision (e.g., 60% precision = 60% WR among predicted wins).
- Plot distribution of Pred_OS on validation set for actual WINs vs LOSSes. Choose the Pred_OS threshold where the WIN distribution significantly exceeds the LOSS distribution.
- Starting values from design: Prob_Win > 0.60 and Pred_OS > 1.1. These are initialization points, not fixed parameters.

Phase 5: Execution Logic

Objective: Translate model outputs into trade decisions with a two-level warmup protocol, a data-validated hard rule for fast bricks, and model inference for normal bricks.

5.1 Two-Level Warmup Protocol

On session start, the system is not immediately ready to trade. Two readiness levels must be reached sequentially:

Level	Condition	Unlocks
Level 1 (Partial)	5 or more bricks have closed since session start AND z-score deques have at least 30 ticks	Hard Rule (Step 2) can fire for fast bricks.
Level 2 (Full)	10 or more bricks have closed AND Micro-Buffer contains 100 ticks	Model Inference (Step 3) can fire for normal bricks.

The binding constraint for Level 2 is the 10-brick history. At median brick duration of 10 minutes, Level 2 is reached approximately 100 minutes after session start. The z-score window will contain thousands of ticks by then, making a separate tick-count warmup condition redundant.

Do Not Trade During Pre-Level 1: Any brick closing before 5 bricks have formed should be skipped entirely. The sequence context is insufficient for the hard rule and the model has no history.

5.2 Step 2: The Hard Rule (Fast Bricks)

Trigger: A brick closes with Duration < 10 seconds.

Check the Sequence Agreement: count how many of the last 5 completed bricks (excluding current) had the SAME direction as the current brick. Call this value Agreement (range 0 to 5).

Agreement	Interpretation	Action	Data WR	Economic EV (after spread)
4 or 5 / 5	Strong trend continuation. Current brick aligns with recent momentum.	Auto-Entry IN brick direction	90.7% (<2s), 92.0% (2-10s)	+0.69 bricks/trade
0 or 1 / 5	Violent reversal. Current brick is a sharp counter-trend breakout.	Auto-Entry IN brick direction (not fade)	86.8% (<2s), 80.1% (2-10s)	+0.55 bricks/trade
2 or 3 / 5	Choppy, indeterminate context.	SKIP. Do not trade.	51.5% (<2s), ~55% (2-10s)	Near-zero or negative after spread

Why Agreement=0 or 1 also triggers entry IN brick direction?: A fast brick that breaks sharply against the recent 5-brick trend represents aggressive counter-trend momentum, not noise. The data confirms these bricks are NOT reversion targets but continuation signals in their own new direction. The key is that Agreement measures alignment with the LAST 5 bricks, and a 0-1 score means the current brick is itself the signal of a new, forceful move.

Hard Rule Boundary Justification (2s vs 10s): The plan previously used 2s. Data analysis shows the sequence agreement signal is equally strong at 5-10s (aligned: 93.1%, opposed: 84.3%, mixed: 61.0%). Mixed WR at 10-20s rises to 74%, making the skip less defensible. Boundary set at 10s to maximize trade count without degrading signal quality.

5.3 Step 3: Model Inference (Normal Bricks)

Trigger: A brick closes with Duration >= 10 seconds AND the system is at Level 2 warmup.

3a. Tensor Construction

- Retrieve the last 10 brick snapshots from Macro-History.
- For each brick: retrieve its stored (100, 9) raw tick snapshot.
- Stack into Micro Tensor: (1, 10, 100, 9).
- Stack the 10 macro-vectors into Macro Tensor: (1, 10, 3).

3b. Inference

- Pass both tensors through the model in a single forward pass (training=False, no dropout active).
- Receive Prob_Win (Head A) and Pred_OS (Head B).

3c. Trade Decision Filters

```
Execute IF: Prob_Win > Threshold_P  AND  Pred_OS > Threshold_M
Starting thresholds: Threshold_P = 0.60,  Threshold_M = 1.1
Final thresholds: calibrated on validation set (see Phase 4.6)
```

The Pred_OS > 1.1 requirement means the model must predict that price will travel more than 1.1x the brick size. This ensures the predicted move covers the TP (1.0x) plus approximate spread cost (0.1x, i.e., ~12% of brick size for XAUUSD).

5.4 Step 4: Order Placement and Exit

Parameter	Value	Rationale
Entry	Market order at brick close	No better entry exists in this framework. The edge is statistical, not timing-based.
Take Profit	Exactly 1 x brick_size from entry	Geometrically consistent across all price levels due to dynamic brick sizing.

Parameter	Value	Rationale
Stop Loss	Exactly 1 x brick_size from entry	1:1 TP/SL. The edge comes entirely from win rate > 50%, not from asymmetric R:R.
Early Exit	None	Do not exit early. The First-Passage-Time probability is the statistical basis of the edge. Cutting winners short destroys it.
Direction	Long if uptrend brick, Short if downtrend brick	Always trade in the direction of the closing brick, regardless of hard rule or model path.

5.5 Complete Execution Flowchart (Per Brick Close)

```
ON EVERY BRICK CLOSE:

 1. Update BrickID, store (100,9) snapshot + M_t in Macro-History

 2. IF system < Level 1 warmup  ->  SKIP. Log and wait.

 3. IF Duration < 10s:

      Compute Agreement = count(last5 bricks same direction as current)

      IF Agreement >= 4  ->  ENTER in brick direction (trend)

      IF Agreement <= 1  ->  ENTER in brick direction (reversal)

      IF Agreement = 2 or 3  ->  SKIP

 4. IF Duration >= 10s AND system at Level 2:

      Build tensor (1,10,100,9) + macro (1,10,3)

      Run model forward pass  ->  Prob_Win, Pred_OS

      IF Prob_Win > 0.60 AND Pred_OS > 1.1  ->  ENTER in brick direction

      ELSE  ->  SKIP

 5. IF entered: set TP = entry +/- 1x brick_size, SL = entry -/+ 1x brick_size
```